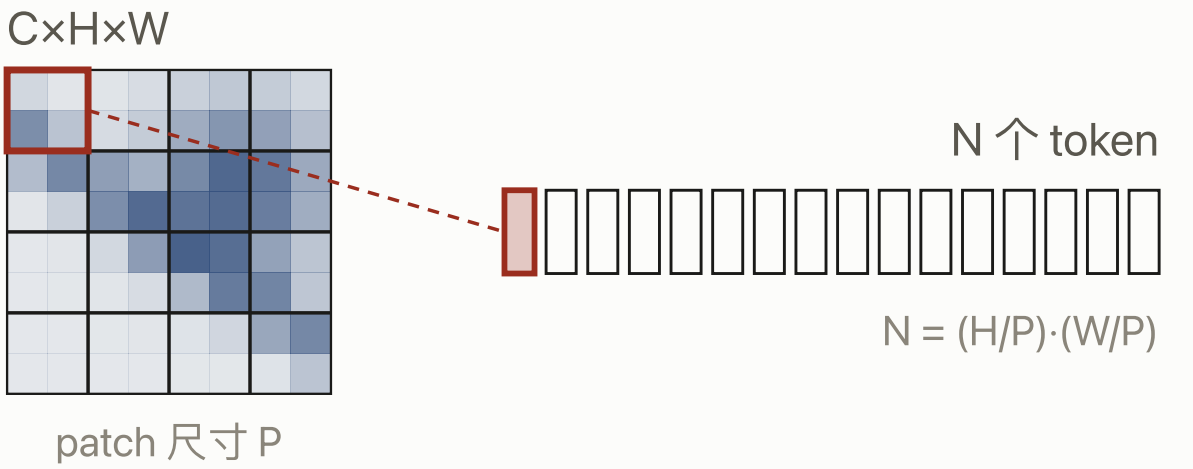
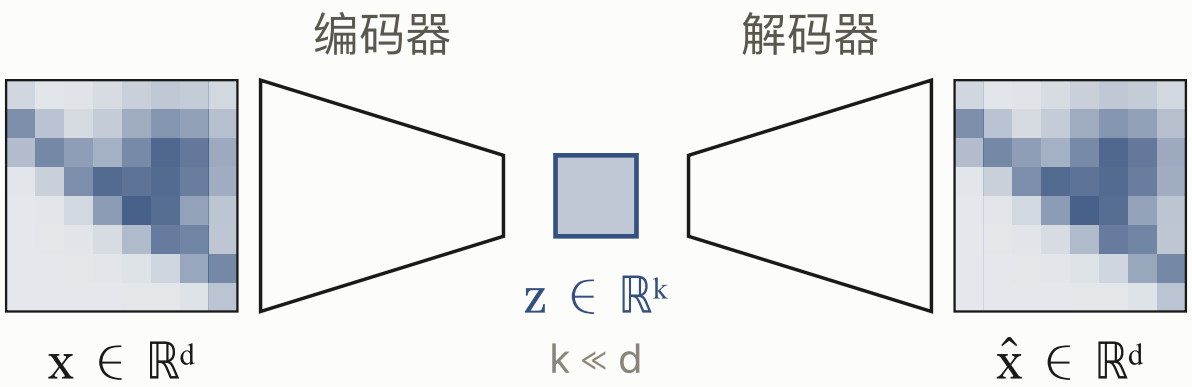
先压小，再生成

图像切成 token 交给 Transformer，或先压进比原图小得多的隐空间





数据空间　　隐空间　　数据空间

目录

01 条件变量的嵌入

1.1 本讲结构与网络的三路输入

1.2 时间嵌入

1.3 类别标签嵌入

1.4 文本输入嵌入

02 扩散 Transformer 与 U-Net

2.1 Patchify 与 DiT 的总体结构

2.2 DiT Block 的数学描述

2.3 U-Net

03 在隐空间中建模：自编码器与 VAE

3.1 为什么要在隐空间建模

3.2 标准自编码器

3.3 变分自编码器：编码器、解码器与损失

3.4 例 31 与 VAE 的先验损失（式 (80)–(83)）

3.5 VAE 的训练

3.6 工程实践说明

3.7 注记 32：在隐空间建模

04 案例研究：Stable Diffusion 3 与 Meta Movie Gen

4.1 Stable Diffusion 3

4.2 Meta Movie Gen Video

在隐空间里做生成：条件嵌入、DiT / U-Net 与 VAE

读完能写出 DiT Block 的计算流程与 VAE 的训练算法，说清为什么要在隐空间里建模，并指出 Stable Diffusion 3 与 Movie Gen 各自用了哪些零件

课程： MIT 6.S184 Flow Matching & Diffusion Models　类型： 个人学习笔记　日期： 2026-10

前几讲讲的是怎么训练流 / 扩散模型，本讲回答怎么把它做大：先把时间和提示词嵌成向量，再用 DiT 或 U-Net 当网络；高分辨率下改在 VAE 的隐空间里建模；最后用 Stable Diffusion 3 和 Movie Gen 两个案例把零件拼起来。

三句话读懂本讲

1. 网络的三路输入先嵌入成向量：时间用傅里叶特征，类别用可学习的嵌入，文本用冻结的预训练模型；主干是 DiT（patchify，加自注意力、交叉注意力与自适应归一化）或 U-Net。
2. 高分辨率下直接在数据空间建模代价太高。用 VAE 把数据压进隐空间，损失 = 重建项 + β·KL 项，靠重参数化训练；必须先训好自编码器，再在隐空间里训练流 / 扩散模型。
3. Stable Diffusion 3 与 Movie Gen 就是这些零件的组合：冻结自编码器的隐空间、条件流匹配、DiT 类网络、多种文本嵌入；视频主要是多出一个时间维度。

01. 条件变量的嵌入

1.1. 本讲结构与网络的三路输入

前面几节，我们学习了如何训练流匹配或扩散模型，实现从分布 $p_{\text{data}}(x|y)$ 采样。这套方案具有通用性，能够适配多种数据类型与应用场景。本节将深入研究**大规模图像与视频生成**的具体实现，会介绍一些知名模型，例如 FLUX 2.0、Stable Diffusion 3、Nano Banana、VEO-3，以及 Meta 的 Movie Gen Video。最后，我们会把前面学到的理论应用到实验中，从零搭建这类模型的简易版本！

本节整体结构安排如下：

1. **神经网络架构**：首先讨论原始条件输入（包括时间 t、原始引导变量 y_{raw}，如离散类别标签或原始文本）如何被转换、编码为向量形式，供模型 $\boldsymbol{u}_t^\theta(x|y)$ 读取。随后介绍 $\boldsymbol{u}_t^\theta(x|y)$ 的主流架构选择，包括 U-Net 与扩散 Transformer（DiT）。
2. **隐空间（Latent Space）**：介绍变分自编码器（VAE）。借助 VAE，我们可以在更低维度的隐空间内做生成建模，从而实现超高分辨率图像生成。
3. **案例研究**：最后深入分析上述两个 SOTA 图像、视频模型（Stable Diffusion、Meta MovieGen），让你理解大规模生成模型的工程实践。

以下对应原讲义的 6.1 节。我们先来面向图像类模态（图像、视频），研究适用于流模型与扩散模型的可扩展神经网络架构。具体来说，我们看带参数 θ 的 **（引导）向量场** $\boldsymbol{u}_t^\theta(x|y)$ 在工程上如何实现。

该神经网络有**三路输入**：向量 $x \in \mathbb{R}^d$、条件变量 $y \in \mathcal{Y}$、时间 $t \in [0,1]$；输出为一个向量 $\boldsymbol{u}_t^\theta(x|y) \in \mathbb{R}^d$。

对于低维分布（比如前面章节的简单测试分布），直接用多层感知机（MLP，全连接网络）对 $\boldsymbol{u}_t^\theta(x|y)$ 做参数化就足够。这种简单场景下，网络前向传播只需要把输入 x, y, t 拼接后送入 MLP。

但对于图像、视频、蛋白质这类复杂高维分布，MLP 性能不足，一般会采用面向特定任务的专用架构。本小节后续以图像为主（视频可同理扩展）。内容分为两部分：

1. 原始条件信息（时间 t、条件变量 y）如何嵌入为模型可处理的向量；
2. 两种主流架构：U-Net，以及扩散 Transformer（DiT）。

1.2. 时间嵌入

以下对应原讲义的 6.1.1 节。对于简单的测试模型，直接将原始标量 t 和输入拼接，就足以训练出效果尚可的网络。但在实际工程中，通常会利用**傅里叶特征（Fourier feature）**将标量时间映射到高维空间，使模型能更精准捕捉高频的时间依赖关系 [46]。

显式特征化公式：

$$\text{TimeEmb}(t) = \Big[\cos(2\pi w_1 t)\cdots\cos(2\pi w_{d/2} t)\ \ \sin(2\pi w_1 t)\cdots\sin(2\pi w_{d/2} t)\Big]^T \tag{68}$$

← 左右滑动查看 →

其中频率 w_i 的设置方式：

$$w_i = w_{\min} \left(\frac{w_{\max}}{w_{\min}} \right)^{\frac{i-1}{d/2-1}}, \quad i = 1, \dots, d/2. \tag{69}$$

TimeEmb 是业界标准方案，但**并非必须严格采用该形式**。它的好处是方便得到归一化的 d 维嵌入向量，即满足 $\|\mathrm{TimeEmb}(t)\| = 1$（利用三角恒等式 $\sin^2 + \cos^2 = 1$）。

校注 · TimeEmb 的范数

原笔记说 $\|\mathrm{TimeEmb}(t)\| = 1$。但式 (68) 的向量里有 $d/2$ 对 $(\cos, \sin)$，每一对的平方和确实是 1（即上面的恒等式），整个 d 维向量的范数却是：

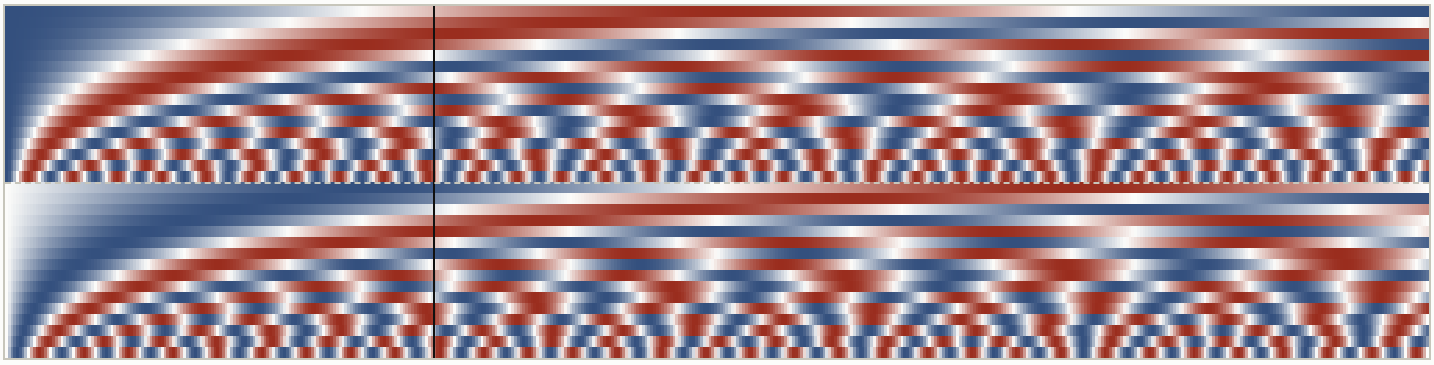
$$\|\mathrm{TimeEmb}(t)\|^2 = \sum_{i=1}^{d/2} \big(\cos^2(2\pi w_i t) + \sin^2(2\pi w_i t) \big) = \frac{d}{2} \implies \|\mathrm{TimeEmb}(t)\| = \sqrt{d}$$

← 左右滑动查看 →

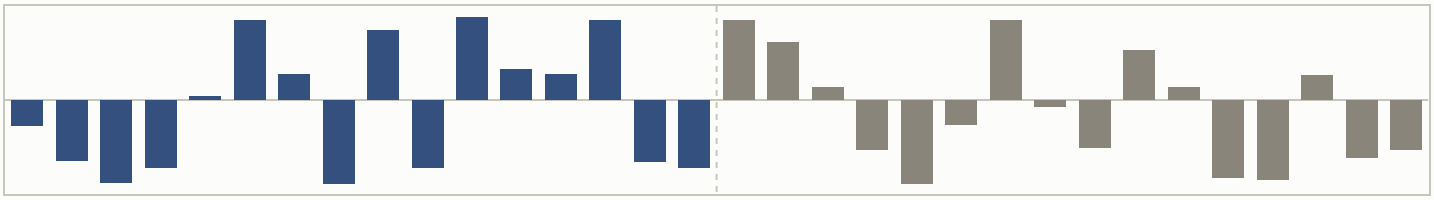
因此“方便得到归一化的向量”要再乘上 $\sqrt{2/d}$ 才严格成立。下面的交互图 A 对任意 d、$w_{\min}$、$w_{\max}$、t 都直接给出这个读数。这只影响“归一化”一词的精确含义，不影响式 (68)(69) 本身。

下面的交互图把式 (68)(69) 画出来（本文补充）：热图的每一列是一个时刻 t 的嵌入向量，低频的行变化缓慢，高频的行变化很快；拖动参数可以看到频率范围如何随 $w_{\min}$、$w_{\max}$ 改变。

热图：横轴 t 从 0 到 1，纵轴是 TimeEmb 的 32 个维度（上半 16 行为 cos，下半 16 行为 sin；每半区内越靠下频率越高）



t = 0.30 处的向量 TimeEmb(t)（左半 cos，右半 sin）



最低频率 w_1 / 最高频率 $w_{d/2}$	1.00 / 32.00
$\|\text{TimeEmb}(t)\|^2$（逐维平方求和）	16.0000　(d/2 = 16)
$\|\text{TimeEmb}(t)\|$ 与 $\sqrt{(d/2)}$	4.0000 与 4.0000
乘以 $\sqrt{(2/d)}$ 之后的范数	1.0000

嵌入维度 d = **32**

最低频率 w_{min} = **1.0**

最高频率 w_{max} = **32**

时间 t = **0.30**

图例：蓝色为正值、红色为负值，颜色越深绝对值越大；竖线标出当前的 t，虚线分开 cos 与 sin 两半。频率按式 (69) 在对数尺度上等比排列（最高频率小于最低频率时按最低频率处理）。读数里的 $\|\text{TimeEmb}(t)\|$ 对任意 t 都等于 $\sqrt{(d/2)}$，不等于 1；乘以 $\sqrt{(2/d)}$ 才得到单位范数。

✦ 交互图 A · 时间嵌入的傅里叶特征（式 (68)(69)）。拖动嵌入维度 d、最低 / 最高频率与时间 t：热图为 TimeEmb(t) 随 t 的变化，柱状图为 t 处的向量，读数给出向量范数与 $\sqrt{(d/2)}$。

1.3. 类别标签嵌入

当原始条件 $y_{\text{raw}} \in \mathcal{Y} \triangleq \{0, \dots, N\}$ 只是类别标签时，最简单的做法是：为 $N+1$ 个可能的类别值各自学习一个独立的嵌入向量，并令 y 等于该嵌入向量。这些嵌入的参数属于网络 $\boldsymbol{u}_t^\theta(x|y)$ 的参数，会在训练过程中一同优化学习。

1.4. 文本输入嵌入

当原始条件 y_{raw} 是文本提示词时，情况会更加复杂，这类方案大多依赖**冻结的预训练模型**。这类预训练模型的作用是把离散文本输入编码为连续向量，用来捕获文本里的语义信息。CLIP（Contrastive Language-Image Pre-training，对比语言-图像预训练）就是这类模型之一。CLIP 的训练目标是构建图像与文本提示词共享的嵌入空间；其损失函数会让图像嵌入和对应的提示词嵌入相互靠近，同时远离不匹配的图像、提示词的嵌入 [34]。因此我们可以令 $y = \text{CLIP}(y_{\text{raw}}) \in \mathbb{R}^{d_{\text{CLIP}}}$，也就是使用冻结的预训练 CLIP 模型输出的向量作为条件。

某些场景下，把整条文本序列压缩成单个向量会丢失信息。这时可以使用预训练 Transformer 对提示词编码，得到**嵌入序列**。在做条件控制时，也经常组合多个不同预训练模型的嵌入，同时利用各个模型的优势 [14, 33]。

在本文的设定中，我们可以简单认为：经过该模型编码后，提示词嵌入的维度为

$$\text{PromptEmbed}(y_{\text{raw}}) \in \mathbb{R}^{S\times k}$$

其中：S 代表 token 序列长度，k 是每个 token 对应的嵌入维度。

02. 扩散 Transformer 与 U-Net

2.1. Patchify 与 DiT 的总体结构

以下对应原讲义的 6.1.2 节。在深入架构细节之前，我们先回顾原讲义引言中的定义：一张图像本质上就是向量 $\boldsymbol{x} \in \mathbb{R}^{C_{\text{image}}\times H\times W}$。其中 C_{image} 代表通道数（RGB 图像一般有 $C_{\text{input}} = 3$ 个颜色通道），H, W 分别是图像像素高度与宽度。

扩散 Transformer（DiT）及其变体是一类非常主流的架构，依靠注意力机制搭建网络 [49, 30, 28]。扩散 Transformer 存在多种变体，这里介绍通用设计；不同模型、不同应用场景下的 DiT 具体实现会存在差异。本节后续记号：

- d：隐空间维度
- L：Transformer 层数
- h：每层注意力头数

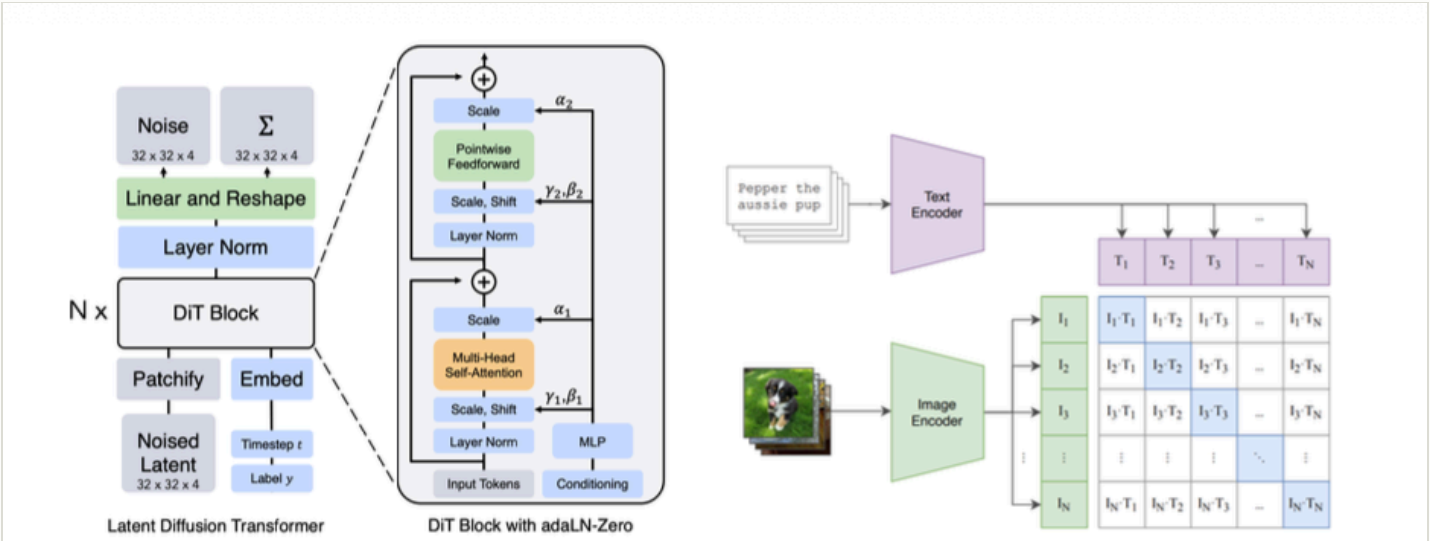


Figure 14: Left: An overview of the diffusion transformer architecture, taken from [30]. Right: A schematic of the contrastive CLIP loss, in which a shared image-text embedding space is learned, taken from [34].

图 14. 左：扩散 Transformer 架构总览，取自 [30]；右：对比式 CLIP 损失的示意，学习图像与文本共享的嵌入空间，取自 [34]。 · 来源：文献 [30]、[34]，MIT 6.S184 讲义

扩散 Transformer 源自视觉 Transformer（ViT），核心思路：将图像切分为图像块（patch），对 patch 做嵌入得到 token 序列，再用标准注意力机制处理 token；最后执行**反分块（depatchification）**操作，还原出原始尺寸的图像。

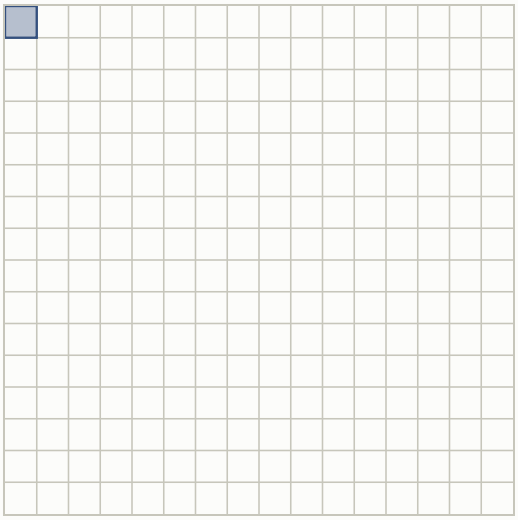
初始分块操作就是对图像张量 $\boldsymbol{x} \in \mathbb{R}^{C \times H \times W}$ 的重构：

$$\text{Patchify}(x) \in \mathbb{R}^{N \times C'}$$

其中 $C' = CP^2$，$N = (H/P) \cdot (W/P)$，P 为 patch 尺寸。接下来对输出做线性变换，得到最终的 patch 嵌入：

$$\text{PatchEmb}(\boldsymbol{x}) = \text{Patchify}(\boldsymbol{x})W \in \mathbb{R}^{N \times d}$$

其中 $W \in \mathbb{R}^{C' \times d}$ 是可学习权重矩阵。下面的交互图把这几个形状算出来（本文补充）：选一个输入张量和 patch 尺寸，就能看到 token 数 N、每个 token 的维度 C' 与自注意力矩阵的大小；切到隐空间的预设，N 会小几个数量级。



每边 16 个 patch（每个 patch 是 2×2 像素，覆盖全部 4 个通道）。左上角高亮的是第 1 个 patch。

量	公式	数值
输入形状	C×H×W	4×32×32
token 数 N	(H/P)·(W/P)	256
每个 token 的维度 C′	C·P²	16
Patchify(x)	N×C′	256×16
PatchEmb(x)	N×d	256×768
可学习权重 W	C′×d	16×768
自注意力矩阵 QK^T 的元素数	N×N	65,536

○ 像素空间 3×1024×1024 ○ FLUX 2.0 隐空间 32×64×64 ◉ Stable Diffusion 隐空间 4×32×32

patch 尺寸 P： ○ 1 ◉ 2 ○ 4 ○ 8 ○ 16 ○ 32

嵌入维度 d = **768**

图例：P 只列出能整除 H 的取值；切换预设时，P 取不超过当前值的最大合法取值。SD 隐空间的预设对应 256×256 的输入（讲义幻灯片：[3,256,256] → [4,32,32]）。切到隐空间预设后，同样 P 下 token 数 N 比像素空间少几个数量级，自注意力的 N×N 矩阵随之缩小；这正是后面“在隐空间里建模”的动机。表中全部是形状与元素数的直接计算，d 只影响 PatchEmb 与 W 的形状。

✦ 交互图 B · Patchify 的形状与 token 数。选择输入张量（像素空间 / FLUX 2.0 隐空间 / Stable Diffusion 隐空间）、patch 尺寸 P 与嵌入维度 d：读数给出 N、C′、Patchify 与 PatchEmb 的形状，以及自注意力矩阵 N×N 的元素数。

扩散 Transformer 的输入包含：时间嵌入、提示词嵌入、经过分块处理后的图像张量（见原讲义 6.1.1 节，即本文第 01 节）：

$$\tilde{\boldsymbol{t}} = \mathrm{TimeEmb}(t) \in \mathbb{R}^d$$
$$\tilde{\boldsymbol{y}} = \mathrm{PromptEmb}(y) \in \mathbb{R}^{S\times d}$$
$$\tilde{\boldsymbol{x}}_0 = \mathrm{PatchEmb}(\boldsymbol{x}) \in \mathbb{R}^{N\times d}$$

可以看到，所有输入都被映射到 Transformer 需要的统一隐维度。

扩散 Transformer 通过多层 DiT Block 迭代更新特征 $\tilde{\boldsymbol{z}}_i$，$i = 0, \ldots, L-1$（细节见注记 29）：

$$\tilde{\boldsymbol{x}}_{i+1} = \mathrm{DiTBlock}(\tilde{\boldsymbol{x}}_i, \tilde{\boldsymbol{t}}, \tilde{\boldsymbol{y}}) \in \mathbb{R}^{N\times d}, \quad (i = 0, \ldots, L-1) \tag{70}$$

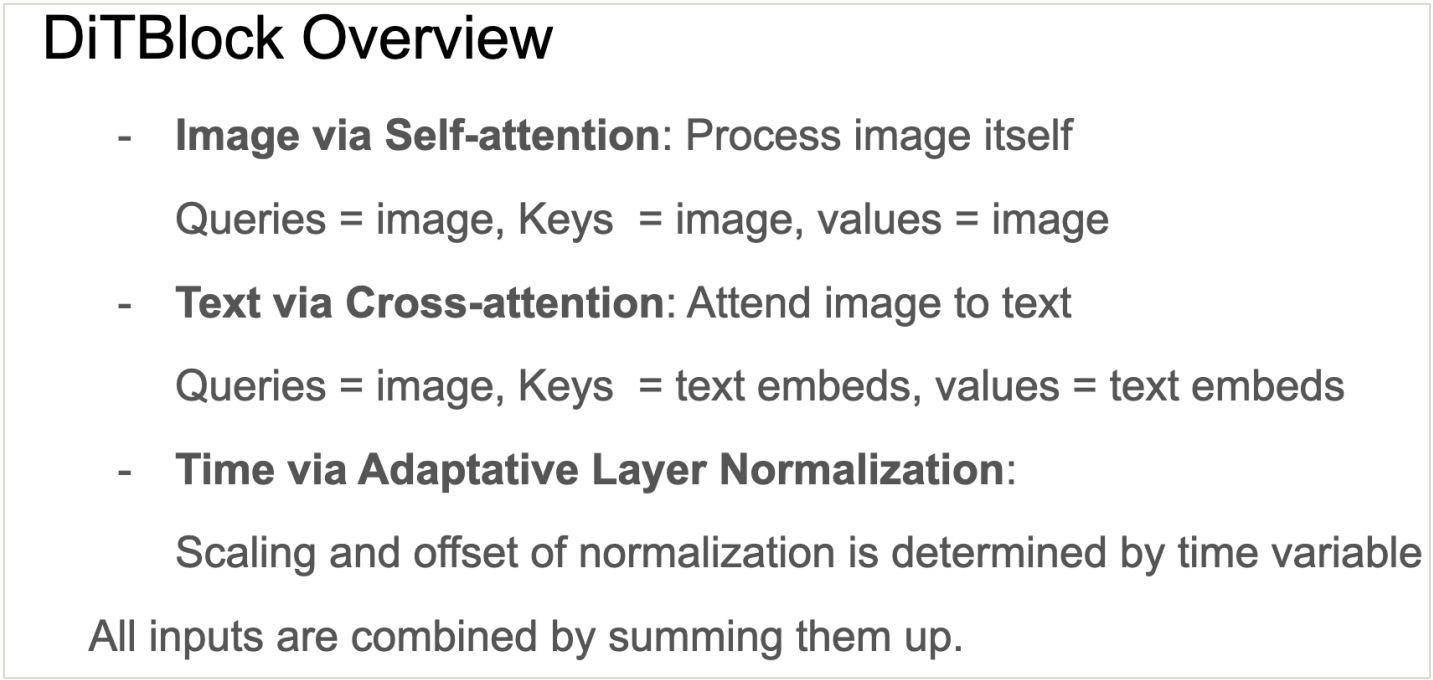
其中 N 为 patch token 的数量。最后执行**反分块（depatchification）**操作，将 DiT 的输出映射回目标输出尺寸：

$$\tilde{\boldsymbol{u}} = \mathrm{Depatchify}\big(\tilde{\boldsymbol{x}}_N W\big) \in \mathbb{R}^{C\times H\times W}$$

式中 $\tilde{W} \in \mathbb{R}^{d\times C'}$。最终得到的张量 $\boldsymbol{u}$ 即为模型输出，也就是预测的速度场 $\boldsymbol{u}_t^\theta(x|y)$。

校注 · 这一小节的符号

原笔记在这里有几处符号不一致，按上下文读即可，不影响理解：迭代的特征在“更新特征 $\tilde{\boldsymbol{z}}_i$”一句里写作 $\tilde{\boldsymbol{z}}_i$，公式里是 $\tilde{\boldsymbol{x}}_i$，指同一个量；层数记作 L，公式 (70) 的下标从 0 到 $L-1$，所以最后一层的输出是 $\tilde{\boldsymbol{x}}_L$（原文写作 $\tilde{\boldsymbol{x}}_N$，其中 N 在别处是 token 数）；反分块的线性层用 $\tilde{W} \in \mathbb{R}^{d\times C'}$（公式里写作 W）。



讲义幻灯片：DiT Block 概览。图像经自注意力处理（Q、K、V 都来自图像）；文本经交叉注意力（Q 来自图像，K、V 来自文本嵌入）；时间经自适应层归一化，归一化的缩放与偏移由时间变量决定；所有输入通过相加结合。 · 来源：MIT 6.S184 讲义

2.2. DiT Block 的数学描述

注记 29 · DiT Block

为完整起见，这里给出单层 DiT 模块的简要数学描述。我们会提供足够细节帮助理解 DiT 模型族，但本文重点是核心算法设计，而非完整的架构工程细节。
设 $\boldsymbol{x} \in \mathbb{R}^{N \times d}$ 为当前 patch token 序列（此处 $\boldsymbol{x} = \tilde{\boldsymbol{x}}_i$），$\boldsymbol{y} \in \mathbb{R}^{S \times d}$ 为条件引导变量的嵌入（此处 $\boldsymbol{y} = \tilde{\boldsymbol{y}}$）。一个标准 DiT 块会通过以下三部分更新 $\boldsymbol{x}$：(i) patch 的自注意力；(ii) 与提示词的交叉注意力；(iii) 基于自适应归一化（AdaLN）的时间条件注入。

缩放点积注意力（Scaled Dot Product Attention）
给定查询 $\boldsymbol{Q} \in \mathbb{R}^{N \times d_h}$、键 $\boldsymbol{K} \in \mathbb{R}^{M \times d_h}$、值 $\boldsymbol{V} \in \mathbb{R}^{M \times d_h}$：

$$\mathrm{Attn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_h}}\right)\boldsymbol{V} \in \mathbb{R}^{N \times d_h}$$

其中 softmax 按行计算。

多头注意力（Multi-Head Attention）
记 h 为注意力头数，$d_h = \dfrac{d}{h}$ 为单头维度。对每个头 $h \in \{1, \dots, n_{\text{heads}}\}$，学习投影矩阵 $\boldsymbol{W}_Q^{(h)}, \boldsymbol{W}_K^{(h)}, \boldsymbol{W}_V^{(h)} \in \mathbb{R}^{k \times d_h}$。定义：

$$\mathrm{head}_h(\boldsymbol{x}, \boldsymbol{z}) = \mathrm{Attn}\big(\boldsymbol{x}\boldsymbol{W}_Q^{(h)}, \boldsymbol{z}\boldsymbol{W}_K^{(h)}, \boldsymbol{z}\boldsymbol{W}_V^{(h)}\big)$$

其中源序列 $\boldsymbol{z}$ 二选一：$\boldsymbol{z} = \boldsymbol{x}$（patch 自注意力），$\boldsymbol{z} = \boldsymbol{y}$（与提示词的交叉注意力）。
拼接所有头的输出，再作用输出投影矩阵 $\boldsymbol{W}_O \in \mathbb{R}^{d \times d}$：

$$\mathrm{MultiHeadAttention}(\boldsymbol{x}, \boldsymbol{z}) = \mathrm{Concat}\big(\mathrm{head}_1(\boldsymbol{x}, \boldsymbol{z}), \dots, \mathrm{head}_h(\boldsymbol{x}, \boldsymbol{z})\big)\boldsymbol{W}_O \in \mathbb{R}^{N \times d}$$

← 左右滑动查看 →

基于自适应归一化的时间条件注入
$\tilde{\boldsymbol{t}} \in \mathbb{R}^d$ 是时间步嵌入。DiT 的标准做法：利用 $\tilde{\boldsymbol{t}}$ 生成逐通道的缩放 / 偏移参数，对归一化后的特征做调制 [31]。具体来说，令 $g : \mathbb{R}^d \to \mathbb{R}^{2d}$ 为 MLP，设

$$(\boldsymbol{\gamma}, \boldsymbol{\beta}) = g(\tilde{\boldsymbol{t}})$$

$\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$（部分实现会为注意力子层、MLP 子层配置独立的 $(\boldsymbol{\gamma}, \boldsymbol{\beta})$）。给定 token 矩阵 $\boldsymbol{x} \in \mathbb{R}^{N \times d}$ 和归一化算子 $\mathrm{Norm}(\cdot)$（如层归一化 LayerNorm），定义调制归一化：

$$\mathrm{AdaNorm}_{\tilde{\boldsymbol{t}}}(\boldsymbol{x}) = (1 + \boldsymbol{\gamma}) \odot \mathrm{Norm}(\boldsymbol{x}) + \boldsymbol{\beta}$$

$\odot$ 代表逐元素相乘，会在 token 维度上广播。

模块整体组合

整合所有操作，得到 DiT Block 的完整计算流程：

$$
\begin{aligned}
\tilde{\boldsymbol{x}} &\leftarrow \boldsymbol{x} + g_{\text{self}}(\tilde{\boldsymbol{t}}) \odot \mathrm{MultiHeadAttention}\big(\mathrm{AdaNorm}_{\tilde{\boldsymbol{t}}}(\boldsymbol{x}), \mathrm{AdaNorm}_{\tilde{\boldsymbol{t}}}(\boldsymbol{x})\big) \\
\tilde{\boldsymbol{x}} &\leftarrow \boldsymbol{x} + g_{\text{cross}}(\tilde{\boldsymbol{t}}) \cdot \mathrm{MultiHeadAttention}\big(\mathrm{AdaNorm}_{\tilde{\boldsymbol{t}}}(\boldsymbol{x}), \boldsymbol{y}\big) \\
\tilde{\boldsymbol{x}} &\leftarrow \boldsymbol{x} + g_{\text{MLP}}(\tilde{\boldsymbol{t}}) \cdot \mathrm{MLP}\big(\mathrm{AdaNorm}_{\tilde{\boldsymbol{t}}}(\boldsymbol{x})\big)
\end{aligned}
$$

其中 MLP 是逐位置前馈网络；$g_{\cdots}$ 是可学习门控参数。输出 $\boldsymbol{x} \in \mathbb{R}^{N \times d}$ 作为下一层的 patch-token 序列（本文记号中即 $\tilde{\boldsymbol{x}}_{i+1}$）。

最后补充：**类别条件 DiT**（实验室实现版本）通常结构更简单，会舍弃交叉注意力层，直接依靠时间 + 类别信息通过 AdaNorm 做条件调制。

校注 · 整合公式的读法

整合公式的三行依次执行，每一行的右侧应使用上一行刚更新的 $\tilde{\boldsymbol{x}}$（原笔记三行右侧都写作 $\boldsymbol{x}$）；第一行的门控用 $\odot$，第二、三行写作 $\cdot$，按逐元素相乘（带广播）理解。另外，原笔记在多头注意力里把头数与头的索引都记作 h，投影矩阵的行数 k 是提示词 token 的嵌入维度，当提示词嵌入与 $\tilde{\boldsymbol{y}} \in \mathbb{R}^{S \times d}$ 的维度一致时 $k = d$。

2.3. U-Net

以下对应原讲义的 6.1.3 节。U-Net 架构 [38] 是 DiT 之外的另一类可选网络结构，属于卷积神经网络的一种。它最初为图像分割任务设计，核心特点是**输入与输出均为图像形状（通道数可以不同）**。这一特性非常适合用来参数化向量场 $\boldsymbol{x} \mapsto \boldsymbol{u}_t^\theta(\boldsymbol{x}|y)$：对于固定的 y, t，输入是图像张量，输出同样是图像形状张量。因此 U-Net 在早期扩散模型文献中被广泛使用 [17, 22, 11]。

U-Net 由一系列编码器 E_i、对应的解码器 D_i，以及位于中间的隐空间处理模块（本文称为 midcoder，中间模块）共同构成。

举个例子，我们以一张图像 $\boldsymbol{x}_t \in \mathbb{R}^{3 \times 256 \times 256}$（输入通道、高、宽取 $(C_{\text{input}}, H, W) = (3, 256, 256)$）在 U-Net 中的前向传播路径来说明：

$$
\begin{aligned}
\boldsymbol{x}_t^{\text{input}} &\in \mathbb{R}^{3 \times 256 \times 256} && \triangleright \text{U-Net 的输入} \\
\boldsymbol{x}_t^{\text{latent}} &= E\big(\boldsymbol{x}_t^{\text{input}}\big) \in \mathbb{R}^{512 \times 32 \times 32} && \triangleright \text{经过编码器得到隐特征} \\
\boldsymbol{x}_t^{\text{latent}} &= M\big(\boldsymbol{x}_t^{\text{latent}}\big) \in \mathbb{R}^{512 \times 32 \times 32} && \triangleright \text{隐特征送入中间模块处理} \\
\boldsymbol{x}_t^{\text{output}} &= D\big(\boldsymbol{x}_t^{\text{latent}}\big) \in \mathbb{R}^{3 \times 256 \times 256} && \triangleright \text{经过解码器得到输出}
\end{aligned}
$$

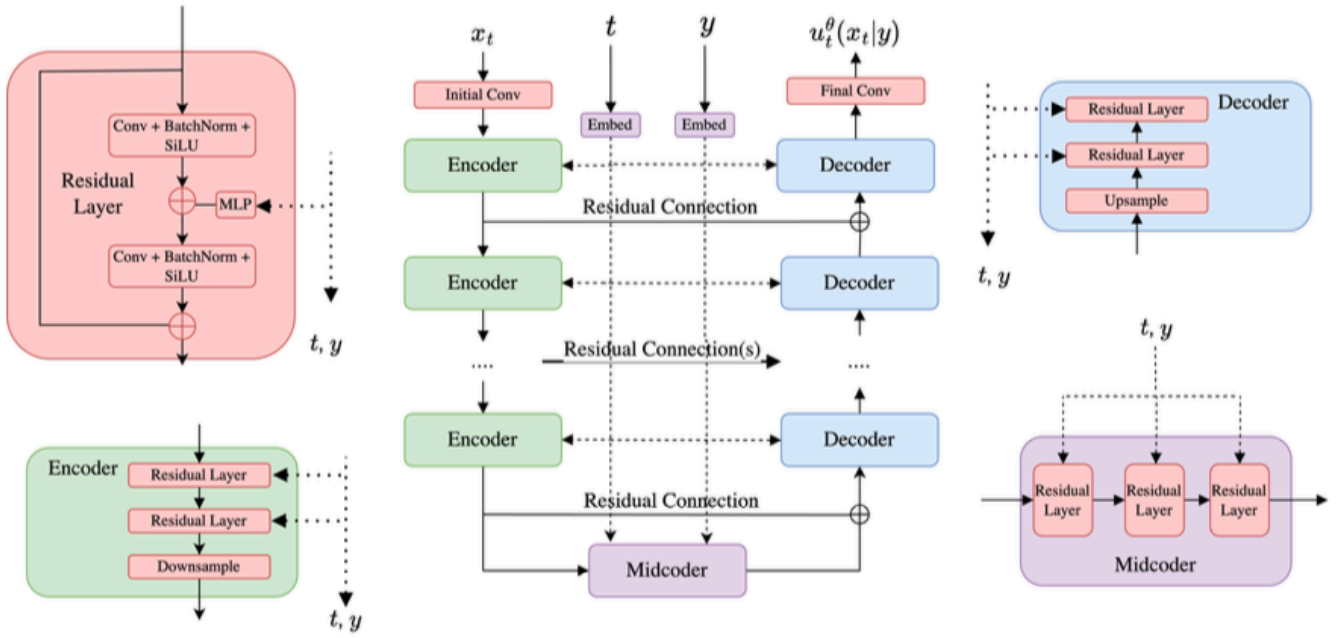


Figure 15: A simplified U-Net architecture (an architecture like this was used in lab 03 of the 2025 version of this course).

图 15. 简化的 U-Net 架构（2025 年版课程的 lab 03 用的就是类似的架构）。 · 来源：MIT 6.S184 讲义

可以观察到：输入经过编码器时，特征的**通道数上升，图像高、宽尺寸不断缩小**。编码器、解码器一般由多层卷积构成，层间附带激活函数、池化等操作。

上面的简化流程省略了两点：

1. 输入 $\boldsymbol{x}_t^{\text{input}} \in \mathbb{R}^{3\times256\times256}$ 通常先经过一个预编码模块提升通道数，再送入第一个编码器块；
2. 编码器与解码器之间一般存在**残差跳跃连接（skip connection）**，完整结构见图 15。

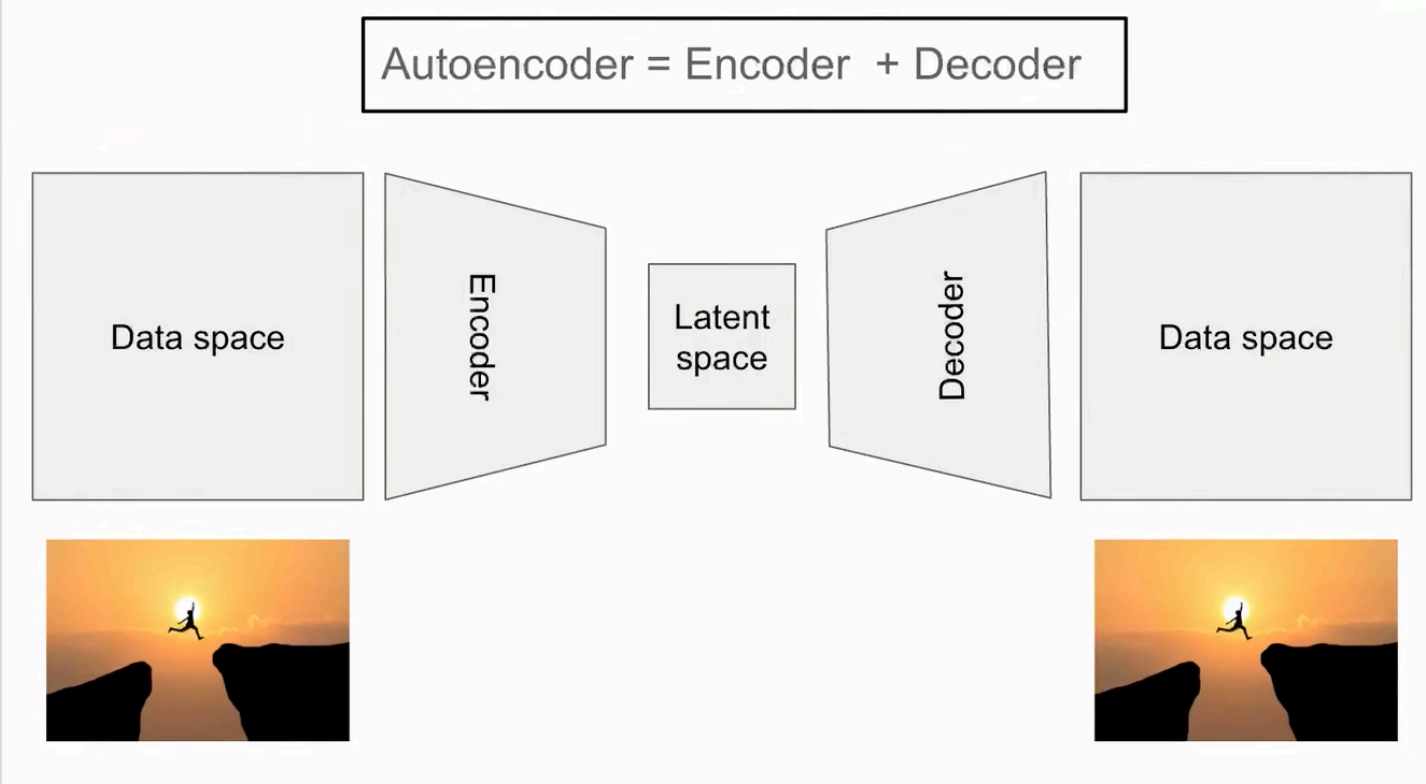
从宏观上看，绝大多数 U-Net 都是上述结构的变体；但实际工程实现中，部分设计会有差异。上面示例是纯卷积架构，而现在常见做法会在编码器、解码器中嵌入注意力层。U-Net 的名字来源于编码器 + 解码器形成的 U 形结构（见图 15）。

03. 在隐空间中建模：自编码器与 VAE

3.1. 为什么要在隐空间建模

以下对应原讲义的 6.2 节。到目前为止，我们一直在数据空间 $\mathbb{R}^d$ 内进行建模。但当图像分辨率持续提高时，直接在原始数据空间建模的开销会迅速难以承受。例如，一张 1024×1024 的 RGB 三通道图像，总维度 $d = H \cdot W \cdot 3 \approx 3 \times 10^6$！对于视频而言，维度会随帧数 T 进一步增大，在如此高维空间中训练很快就会变得不可行。

图像分类任务的输出是低维向量，可以逐步缩小卷积网络的特征尺寸；而基于流的建模方法不一样，它要求输出 $\boldsymbol{u}_t^\theta(\boldsymbol{x}) \in \mathbb{R}^d$ 和输入拥有完全相同的维度。因此一个关键问题是：**如何在有限的显存与计算预算下对高维图像建模?**



讲义幻灯片：自编码器 = 编码器 + 解码器。图像经编码器压缩进隐空间，再由解码器还原回数据空间。 · 来源：MIT 6.S184 讲义

3.2. 标准自编码器

以下对应原讲义的 6.2.1 节。一个很自然的思路是压缩：图像的真实流形，其实位于高维图像空间内一个低维流形附近。更严格地说，我们可以定义编码器 $\mu_\phi : \mathbb{R}^d \to \mathbb{R}^k$ 与解码器 $\mu_\theta : \mathbb{R}^k \to \mathbb{R}^d$，二者配合完成原始图像 $\boldsymbol{x} \in \mathbb{R}^d$ 和隐变量 $\boldsymbol{z} \in \mathbb{R}^k$ 之间的双向映射。隐空间维度 k 通常远小于原始数据维度 d。以图像为例，若 $d = 3 \times 1024 \times 1024$，常见做法是下采样，取 $k = 3 \times \frac{1024}{16} \times \frac{1024}{16}$。

μ_ϕ 和 μ_θ 合称为**自编码器（Autoencoder）**。理想情况下，编码器与解码器要保证很高的重建质量：也就是平均来看 $\mu_\theta(\mu_\phi(\boldsymbol{x}))$ 和 $\boldsymbol{x}$ 尽可能接近。自编码器一般使用重建损失训练：

$$\mathcal{L}_{\text{Recon}}(\phi,\theta)=\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\left\|\mu_\theta(\mu_\phi(\boldsymbol{x}))-\boldsymbol{x}\right\|^2$$

该损失衡量原始样本 $\boldsymbol{x}$ 与重建样本 $\mu_\theta(\mu_\phi(\boldsymbol{x}))$ 之间的均方误差。

生成建模的适配性

遗憾的是，仅靠上述重建损失不足以训练出“好用”的自编码器。我们最终目标是在隐空间训练生成模型，拟合隐分布 $p_{\text{latent}}(\boldsymbol{z})$，其中 $\boldsymbol{z}=\mu_\phi(\boldsymbol{x}),\boldsymbol{x}\sim p_{\text{data}}$。之后，把隐空间生成模型的输出送入解码器 μ_θ，就能得到原始数据分布 $p_{\text{data}}(\boldsymbol{x})$。

但上述基础自编码器存在一个微妙缺陷：我们几乎无法控制隐分布 $p_{\text{latent}}(\boldsymbol{z})$，也就无法保证这个分布形态足够“友好”，适合训练生成模型（例如平滑、简单、近似高斯分布）。虽然隐空间实现了数据压缩，但原始数据分布 p_{data} 可能被映射成一个极难学习的隐分布 p_{latent}。于是问题变为：如何保证隐分布 p_{latent} 形态良好、易于学习？为了对隐分布施加更显式的正则约束，我们把自编码器放到更通用的概率框架下，由此引出**变分自编码器（VAE）**。

3.3. 变分自编码器：编码器、解码器与损失

以下对应原讲义的 6.2.2 节。变分自编码器（VAE）是在（确定性）标准自编码器基础上，**解除编码器、解码器必须为确定性函数**这一约束得到的概率模型。具体来说，设参数为 ϕ 的编码器 $q_\phi(\boldsymbol{z}|\boldsymbol{x})$，以及参数为 θ 的解码器 $p_\theta(\boldsymbol{x}|\boldsymbol{z})$。最常用设定为高斯分布：

$$q_\phi(\boldsymbol{z}|\boldsymbol{x})=\mathcal{N}\big(\boldsymbol{z};\boldsymbol{\mu}_\phi(\boldsymbol{x}),\operatorname{diag}(\sigma^2_\phi(\boldsymbol{x}))\big),\quad p_\theta(\boldsymbol{x}|\boldsymbol{z})=\mathcal{N}\big(\boldsymbol{x};\boldsymbol{\mu}_\theta(\boldsymbol{z}),\sigma^2_\theta(\boldsymbol{z})\boldsymbol{I}_d\big)\tag{71}$$

← 左右滑动查看 →

式中 $\boldsymbol{\mu}_\phi(\boldsymbol{x})\in\mathbb{R}^k,\sigma^2_\phi(\boldsymbol{x})\in\mathbb{R}^k_{\geq 0},\boldsymbol{\mu}_\theta(\boldsymbol{z})\in\mathbb{R}^d,\sigma^2_\theta(\boldsymbol{z})\in\mathbb{R}_{\geq 0}$ 均由神经网络参数化；$\operatorname{diag}(\cdot)$ 代表对角矩阵。

编码、解码时需要采样：

$$\begin{aligned}\boldsymbol{z}&\sim q_\phi(\cdot|\boldsymbol{x})\quad(\text{编码})\\ \boldsymbol{x}&\sim p_\theta(\cdot|\boldsymbol{z})\quad(\text{解码})\end{aligned}$$

可以看到：若恒有 $\sigma_\phi(\boldsymbol{x})=0,\sigma_\theta(\boldsymbol{x})=0$，VAE 就退化为标准自编码器。

下面看重建损失。VAE 的重建目标定义为：

$$\mathcal{L}_{\text{VAE-Recon}}(\phi,\theta) = -\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{z}\sim q_\phi(\cdot|\boldsymbol{x})}\big[\log p_\theta(\boldsymbol{x}|\boldsymbol{z})\big] \tag{72}$$

两处关键变化：不再是确定性编码，而是从 $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ 采样隐变量 $\boldsymbol{z}$；损失取解码分布下 $\boldsymbol{x}$ 的负对数似然。直观理解：把编码、解码的随机性全部纳入考量，衡量“经过随机编码再解码后，原始数据 $\boldsymbol{x}$ 的对数概率”。

对于高斯形式，重建损失可展开为：

$$\mathcal{L}_{\text{VAE-Recon}}(\phi,\theta) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{z}\sim q_\phi(\boldsymbol{z}|\boldsymbol{x})}\left[\frac{1}{2\sigma_\theta^2(\boldsymbol{z})}\big\|\boldsymbol{x}-\boldsymbol{\mu}_\theta(\boldsymbol{z})\big\|^2 + \frac{d}{2}\log\sigma_\theta^2(\boldsymbol{z}) + \text{const}\right]$$

← 左右滑动查看 →

该式利用正态分布概率密度推导（见前几讲的式 (97)）。VAE 重建损失和普通 AE 重建损失本质相近，只是需要对所有可能的编码采样 $\boldsymbol{z}\sim q_\phi(\cdot|\boldsymbol{x})$ 求期望。依赖解码器方差的第二项，用来平衡重建精度与预测不确定性。

包括实验室实现在内的很多工程代码，会将 $\sigma_\phi(\boldsymbol{x})$、$\sigma_\theta(\boldsymbol{z})$ 固定为可学习的标量常数（分别和 $\boldsymbol{x},\boldsymbol{z}$ 无关），避免学习方差时出现病态行为、提升数值稳定性。此时 VAE 重建损失在编码随机性与常数项之外，基本等价于标准自编码器重建损失：

$$\mathcal{L}_{\text{VAE-Recon}}(\phi,\theta) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{z}\sim q_\phi(\boldsymbol{z}|\boldsymbol{x})}\left[\frac{1}{2\sigma_\theta^2}\big\|\boldsymbol{x}-\boldsymbol{\mu}_\theta(\boldsymbol{z})\big\|^2 + \text{const}\right] \tag{74}$$

回到我们的目标：对数据分布 $p_{\text{data}}(\boldsymbol{x})$ 做编码，映射到隐空间后得到形态“友好”、易于学习的分布。为此，我们在隐变量 $\boldsymbol{z}$ 上引入先验分布 $p_{\text{prior}}(\boldsymbol{z})$。本文取**各向同性高斯分布** $p_{\text{prior}}=\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k)$。这个先验代表隐分布的理想形态：高斯分布简单、易于拟合，满足我们对“可训练隐分布”的需求。

核心思想：对编码器施加正则，让编码后的分布尽可能贴近先验分布 p_{prior}，通过下面的辅助损失实现：

$$\mathcal{L}_{\text{VAE-Prior}}(\phi) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\big[D_{\text{KL}}\big(q_\phi(\cdot|\boldsymbol{x})\,\|\,p_{\text{prior}}\big)\big] \tag{75}$$

D_{KL} 为 KL 散度（Kullback-Leibler divergence），是衡量两个概率分布差异的基础工具。详细推导超出本文范围，注记 30 会做简要回顾。

损失 $\mathcal{L}_{\text{VAE-Prior}}$ 的物理含义很直观：对任意样本 $\boldsymbol{x}$，希望它对应的编码分布尽量接近高斯分布。如果所有样本都满足该约束，那么整体隐分布自然趋近高斯。

注记 30 · KL 散度背景

给定两个概率密度 q, p，KL 散度定义为：

$$D_{\mathrm{KL}}\big(q(\boldsymbol{x}) \,\|\, p(\boldsymbol{x})\big) = \int q(\boldsymbol{x}) \log \frac{q(\boldsymbol{x})}{p(\boldsymbol{x})} d\boldsymbol{x} := \mathbb{E}_{X\sim q}\left[\log \frac{q(X)}{p(X)}\right].$$

KL 散度是衡量分布之间差异的标准指标，有如下重要性质：

$$D_{\mathrm{KL}}(q \,\|\, p) \geq 0, \tag{76}$$

$$D_{\mathrm{KL}}(q \,\|\, p) = 0 \iff q = p. \tag{77}$$

即 KL 散度恒非负；当且仅当两个分布完全相同时，KL 散度等于 0。

为定义变分自编码器的损失函数，我们将重建损失与先验正则损失结合，并引入权重系数 $\beta \geq 0$，得到 VAE 训练目标：

$$\begin{aligned}\mathcal{L}_{\text{VAE}}(\phi, \theta) &= \mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) + \beta\, \mathcal{L}_{\text{VAE-Prior}}(\phi) \\ &= -\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}, \boldsymbol{z}\sim q_\phi(\boldsymbol{z}|\boldsymbol{x})}\big[\log p_\theta(\boldsymbol{x}|\boldsymbol{z})\big] + \beta\, \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\big[D_{\mathrm{KL}}\big(q_\phi(\cdot|\boldsymbol{x}) \,\|\, p_{\text{prior}}\big)\big]\end{aligned} \tag{7}$$

← 左右滑动查看 →

第一项约束隐变量能够被有效地解码还原为原始数据；第二项约束隐分布尽量接近高斯分布。参数 β 用来控制两项损失的相对强度。

3.4. 例 31 与 VAE 的先验损失（式 (80)–(83)）

下面针对高斯分布场景推导 KL 散度的具体表达式：

例 31 · 对角协方差高斯分布之间的 KL 散度

设 $q(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_q, \mathrm{diag}(\boldsymbol{\sigma}_q^2))$，$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_p, \mathrm{diag}(\boldsymbol{\sigma}_p^2))$ 为对角协方差高斯分布，$\boldsymbol{\sigma}_q, \boldsymbol{\sigma}_p \in \mathbb{R}^d_{\geq 0}$，$\boldsymbol{x} \in \mathbb{R}^d$。则

$$D_{\mathrm{KL}}(q \,\|\, p) = \frac{1}{2} \sum_{i=1}^{d} \left(\frac{\sigma_{q,i}^2}{\sigma_{p,i}^2} + \frac{(\mu_{q,i} - \mu_{p,i})^2}{\sigma_{p,i}^2} - \log \frac{\sigma_{q,i}^2}{\sigma_{p,i}^2} - 1 \right) \tag{80}$$

上式物理含义直观：当均值与方差完全相等时，$D_{\mathrm{KL}}(q \,\|\, p) = 0$；KL 散度会随均值向量的平方误差 $\|\boldsymbol{\mu}_q - \boldsymbol{\mu}_p\|^2$ 增大而上升。函数的每一分量在 $\sigma_{q,i} = \sigma_{p,i}$（即 $\alpha_i = 1$）处取得极小值，因此当 $\boldsymbol{\sigma}_q = \boldsymbol{\sigma}_p$ 时 KL 散度达到最小。

证明：我们先证明一维情形（$d = 1$）；高维 $d > 1$ 可对各维度求和，推导方式完全一致。由正态分布的概率密度（见前几讲的式 (97)）：

$$\log q(x) = -\frac{1}{2}\log(2\pi\sigma_q^2) - \frac{1}{2\sigma_q^2}(x - \mu_q)^2, \quad \log p(x) = -\frac{1}{2}\log(2\pi\sigma_p^2) - \frac{1}{2\sigma_p^2}(x - \mu_p)$$

← 左右滑动查看 →

KL 散度定义：

$$D_{\mathrm{KL}}(q \,\|\, p) = \mathbb{E}_{x \sim q}\big[\log q(x) - \log p(x)\big]$$

当 $x \sim \mathcal{N}(\mu_q, \sigma_q^2 I)$：

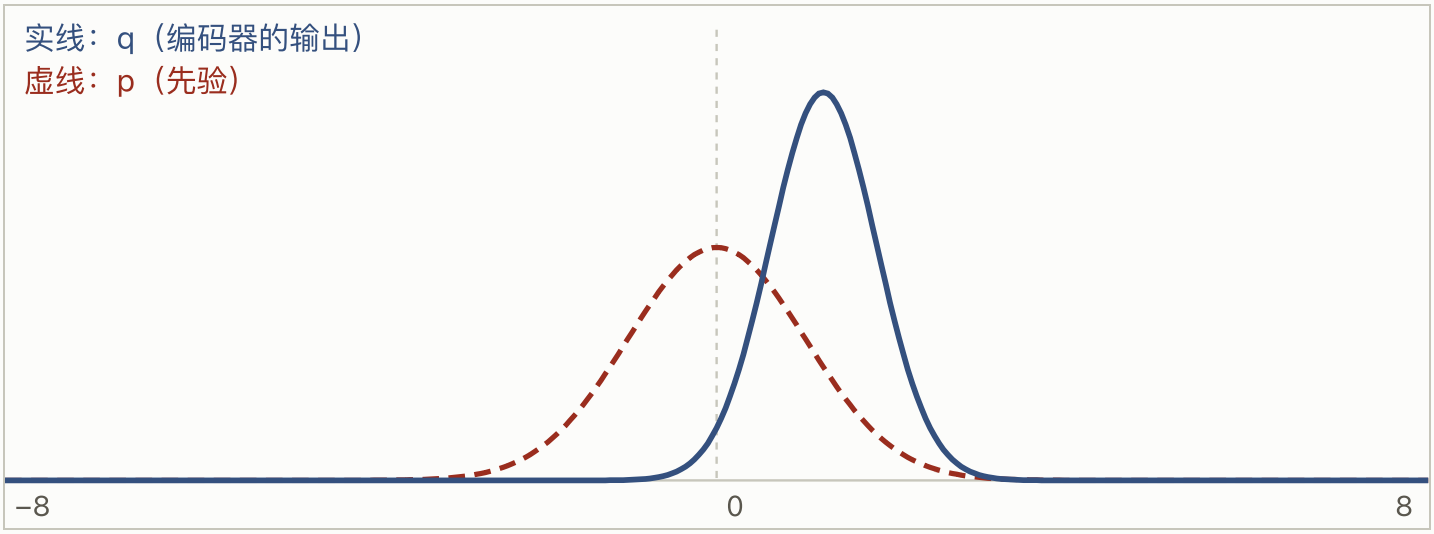
$$\begin{aligned} D_{\mathrm{KL}}(q \,\|\, p) = \frac{1}{2}\log\frac{\sigma_p^2}{\sigma_q^2} + \frac{1}{2\sigma_p^2}\mathbb{E}_q\left[(x - \mu_p)^2\right] \\ - \frac{1}{2\sigma_q^2}\mathbb{E}_q\left[(x - \mu_q)^2\right] \end{aligned} \tag{81}$$

其中 $\mathbb{E}_q\left[(x - \mu_q)^2\right] = \sigma_q^2$。利用分解：$x - \mu_p = (x - \mu_q) + (\mu_q - \mu_p)$，且 $\mathbb{E}_q[x - \mu_q] = 0$，可得：

$$\mathbb{E}_q\left[(x - \mu_p)^2\right] = \mathbb{E}_q\left[(x - \mu_q)^2\right] + (\mu_q - \mu_p)^2 = \sigma_q^2 + (\mu_q - \mu_p)^2$$

将上述结果代入式 (81)，即可得到式 (80)。

下面的交互图把式 (80) 的四项逐项算出来，并与对 $q(\log q - \log p)$ 做数值积分的结果对照（本文补充）。



σ_q^2/σ_p^2	0.3600	$(\mu_q - \mu_p)^2/\sigma_p^2$	1.4400
$-\log(\sigma_q^2/\sigma_p^2)$	1.0217	−1	-1.0000
式 (80)：½ × 四项之和		0.9108	
数值积分 ∫ q (log q − log p) dx		0.9108 （与闭式之差 1.3e-15）	
p = N(0, 1) 时式 (82) 的一维写法		0.9108（与闭式相等）	
β·KL（先验正则项，β = 0.50）		0.4554	

q 的均值 μ = **1.20**

q 的标准差 σ = **0.60**

p 的均值 μ = **0.00**

p 的标准差 σ = **1.00**

权重 β = **0.50**

图例：默认的先验 p 是标准高斯 N(0, 1)（把 p 的均值和标准差调回 0 和 1 即可）。q 与 p 的均值、方差完全相同时 KL 为 0；均值差拉大，第二项上升；q 的标准差离 p 的标准差越远，第一项与第三项的和越大，两者相等时最小。数值积分的区间取 q 的均值上下各 10 倍标准差（区间外 q 的质量约 1e-23，可忽略）；密度图只画 [−8, 8]，q 很宽或偏心时曲线会在边缘被截断。

✦ 交互图 C · 两个一维高斯之间的 KL 散度（式 (80)）。拖动 q 与 p 的均值、标准差与权重 β：读数给出式 (80) 的四项、闭式值、数值积分值，以及 β·KL。

现在假设编码器输出为高斯分布形式，则可以得到：

$$\begin{aligned}\mathcal{L}_{\text{VAE-Prior}}(\phi) &= \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\left[D_{\text{KL}}\left(q_\phi(\cdot|\boldsymbol{x}) \parallel \mathcal{N}(\mathbf{0}, \boldsymbol{I}_k)\right)\right] \\ &= \mathbb{E}\left[\frac{1}{2}K\left(\boldsymbol{\sigma}_\phi^2(\boldsymbol{x})\right) + \frac{1}{2}\left\|\boldsymbol{\mu}_\phi(\boldsymbol{x})\right\|^2\right]\end{aligned} \tag{82}$$

该损失物理含义直观：对均值 $\boldsymbol{\mu}_\phi(\boldsymbol{x})$ 施加惩罚，使其尽量趋近 0；对方差施加惩罚，使其尽量趋近 1。

校注 · K(σ²) 的定义

原笔记在式 (82) 与式 (83) 里使用了 $K(\boldsymbol{\sigma}^2)$，但没有给出定义。把先验 $p=\mathcal{N}(\mathbf{0},\boldsymbol{I}_k)$（即 $\mu_{p,i}=0$, $\sigma_{p,i}=1$）代入式 (80)，逐维相加得到：

$$D_{\text{KL}}\left(q_\phi(\cdot|\boldsymbol{x}) \parallel \mathcal{N}(\mathbf{0}, \boldsymbol{I}_k)\right) = \frac{1}{2}\sum_{j=1}^{k}\left(\sigma_{\phi,j}^2 - \log\sigma_{\phi,j}^2 - 1\right) + \frac{1}{2}\|\boldsymbol{\mu}_\phi(\boldsymbol{x})\|^2$$

因此 $K(\boldsymbol{\sigma}^2)=\sum_{j=1}^{k}\left(\sigma_j^2-\log\sigma_j^2-1\right)$，这也正是后面算法 6 第 7 行 KL 损失里对 j 求和的那一项。交互图 C 在 $p=\mathcal{N}(0,1)$ 时给出的一维读数就是这个式子。

VAE 总损失写作：

$$\begin{aligned}\mathcal{L}_{\text{VAE}}(\phi,\theta) &= \mathcal{L}_{\text{VAE-Recon}}(\phi,\theta) + \beta\,\mathcal{L}_{\text{VAE-Prior}}(\phi) \\ &= \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{z}\sim q_\phi(\boldsymbol{z}|\boldsymbol{x})}\Bigg[\underbrace{\frac{1}{2\sigma_\theta^2(\boldsymbol{z})}\left\|\boldsymbol{x}-\boldsymbol{\mu}_\theta(\boldsymbol{z})\right\|^2}_{\text{重建误差}} + \underbrace{\frac{d}{2}\log\sigma_\theta^2(\boldsymbol{z})}_{\text{解码器置信度}} + \underbrace{\frac{\beta}{2}K\left(\boldsymbol{\sigma}_\phi^2(\boldsymbol{x})\right)}_{\text{约束隐变量方差=1}} -\end{aligned}$$

← 左右滑动查看 →

损失函数共四项，含义清晰：

- 第一项就是重建误差；
- 第二项描述解码器的不确定性：方差越小，解码器“置信度”越高，但同时会对重建误差施加更强惩罚；
- 另外两项用来约束隐分布：使隐变量方差趋近 1、均值趋近 0，保证隐分布贴近高斯分布。

3.5. VAE 的训练

下面讨论如何最小化 VAE 损失 $\mathcal{L}_{\text{VAE}}(\phi,\theta)$。这个损失的难点在于：求期望时的分布 $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ 依赖待优化参数 ϕ。我们可以使用**重参数化技巧（reparameterization trick）**改写期望。对于

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}\big(\boldsymbol{z};\boldsymbol{\mu}_\phi(\boldsymbol{x}),\boldsymbol{\sigma}^2_\phi(\boldsymbol{x})\boldsymbol{I}_k\big)$$

采样方式改写为：

$$\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k),\quad \boldsymbol{z}=\boldsymbol{\mu}_\phi(\boldsymbol{x})+\boldsymbol{\sigma}_\phi(\boldsymbol{x})\odot\boldsymbol{\epsilon}\implies \boldsymbol{z}\sim q_\phi(\cdot|\boldsymbol{x})$$

式中唯一的噪声来源是 $\boldsymbol{\epsilon}$，而 $\boldsymbol{\epsilon}$ 的分布与参数 ϕ 相互独立。于是损失可以改写为：

$$\mathcal{L}_{\text{VAE}}(\phi,\theta)=\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k)}\left[\frac{1}{2\sigma^2_\theta(\boldsymbol{z})}\big\|\boldsymbol{x}-\boldsymbol{\mu}_\theta\big(\boldsymbol{\mu}_\phi(\boldsymbol{x})+\boldsymbol{\sigma}_\phi(\boldsymbol{x})\odot\boldsymbol{\epsilon}\big)\big\|^2+\frac{d}{2}\log\sigma^2_\theta(\boldsymbol{z})\right.$$

← 左右滑动查看 →

重参数化之后，随机性仅来自 $\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k)$，该分布不依赖 ϕ。这样就可以用深度学习标准优化器最小化该损失。进一步简化，再次固定 $\sigma^2_\theta(\boldsymbol{z})=\sigma^2$ 为全局常数：

$$\mathcal{L}_{\text{VAE}}(\phi,\theta)=\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}},\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I}_k)}\left[\frac{1}{2\sigma^2}\big\|\boldsymbol{x}-\boldsymbol{\mu}_\theta\big(\boldsymbol{\mu}_\phi(\boldsymbol{x})+\boldsymbol{\sigma}_\phi(\boldsymbol{x})\odot\boldsymbol{\epsilon}\big)\big\|^2+\frac{\beta}{2}K\big(\boldsymbol{\sigma}^2_\phi(\boldsymbol{x})\big)\right.$$

← 左右滑动查看 →

算法 6 总结了 VAE 的训练流程（原讲义以截图给出，此处重排）：

Algorithm 6 β-VAE Training Procedure (Gaussian decoder with fixed variance $p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \tilde{\sigma}^2 I_d)$)

输入：由数据样本 $x \sim p_{\text{data}}$ 构成的数据集，编码器网络 $(\mu_\phi(x), \log \sigma_\phi^2(x))$，解码器网络 $\mu_\theta(z)$，隐空间维度 k，常数 $\beta \geq 0$, $\tilde{\sigma}^2 > 0$

for 每个 mini-batch $\{x_i\}_{i=1}^B$ **do**

  编码每个 x_i：$\mu_i \leftarrow \mu_\phi(x_i)$, $\log \sigma_i^2 \leftarrow \log \sigma_\phi^2(x_i)$

  采样噪声 $\epsilon_i \sim \mathcal{N}(0, I_k)$

  重参数化：$z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$　　其中 $\sigma_i = \exp(\frac{1}{2} \log \sigma_i^2)$

  解码得到均值：$\hat{x}_i \leftarrow \mu_\theta(z_i)$

  重建损失：

$$\mathcal{L}_{\text{recon}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2\tilde{\sigma}^2} \|x_i - \hat{x}_i\|^2$$

  相对先验 $p_{\text{prior}}(z) = \mathcal{N}(0, I_k)$ 的 KL 损失：

$$\mathcal{L}_{\text{KL}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2} \sum_{j=1}^k \left(\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1\right)$$

  总损失：$\mathcal{L} \leftarrow \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$

  更新 $(\phi, \theta) \leftarrow \text{grad_update}(\mathcal{L})$

end for

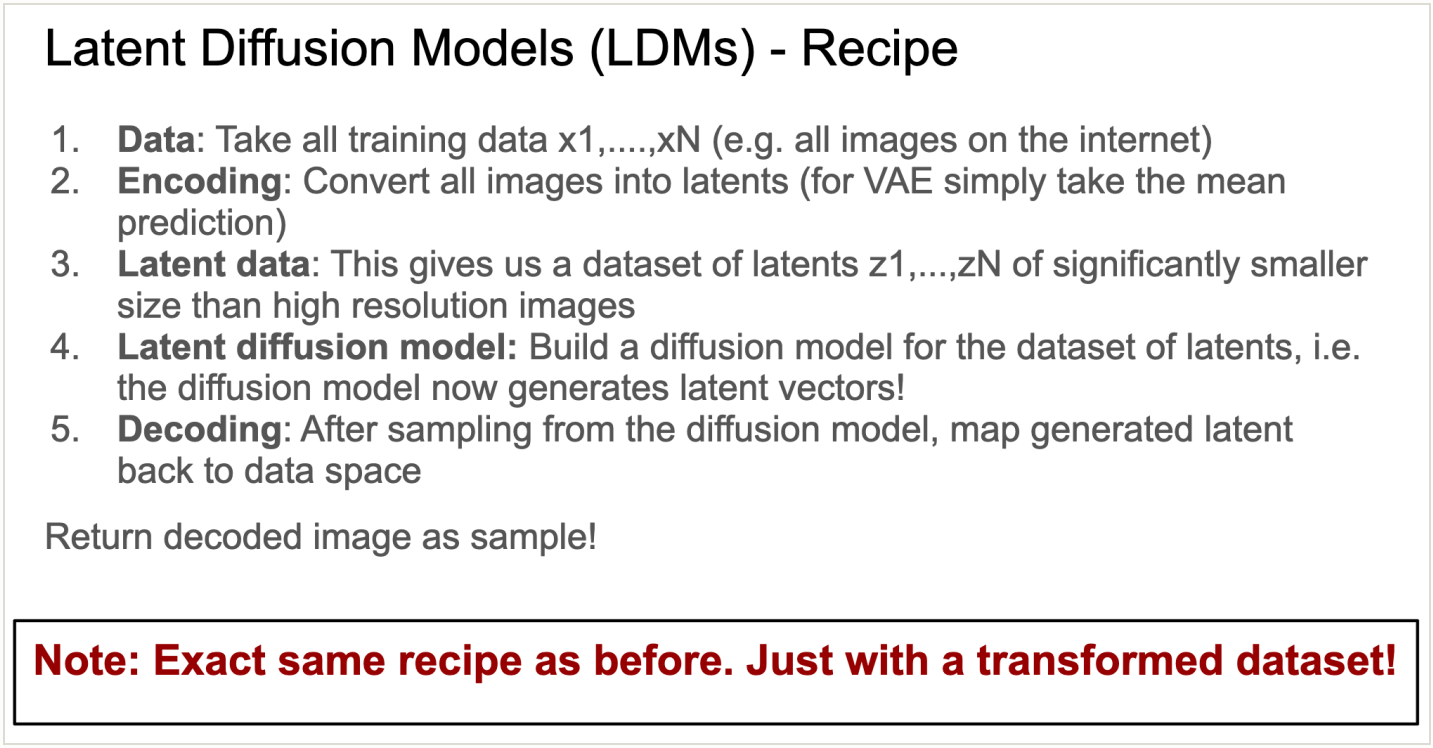
✦ Algorithm 6 · β-VAE 的训练流程（高斯解码器，固定方差；据原讲义重排。原截图的 Require 一行写作 σ²>0，标题写作 σ²，此处统一记作 σ²）。

3.6. 工程实践说明

上面推导展示了自编码器的设计原理。实际工程中，研究者常会增加额外损失项或约束。这里补充几点自编码器工程经验：

1. **β 的选取与 KL 预热（KL warm-up）**：较大的 β 会强制隐分布贴近先验，但会损害重建质量，甚至出现**后验坍缩（posterior collapse）**：编码器直接忽略输入 $\boldsymbol{x}$，输出 $q_\phi(\boldsymbol{z}|\boldsymbol{x}) \approx \mathcal{N}(\mathbf{0}, \boldsymbol{I}_k)$。常用稳定手段是 KL 预热：训练初期令 $\beta = 0$，在前若干个 epoch 中逐步提升到目标值。现代自编码器一般使用很小的 β，即 $\beta \ll 1$。
2. **解码器方差**：学习高斯解码器方差 σ_θ^2 在数值上比较敏感；如果不加正则，容易出现退化解。为保证稳定性，很多实现固定 $p_\theta(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_\theta(\boldsymbol{z}), \sigma^2 \boldsymbol{I}_d)$，令 σ^2 为常数。此时重建项在忽略常数后等价于均方误差。

3. **像素 MSE 之外的重建损失**：对图像而言，逐像素高斯似然（即均方误差）往往会产生过度平滑的重建结果。工程上常会加入**感知损失**（利用预训练网络在特征空间计算损失），提升图像锐度与语义保真度。
4. **对抗式与混合目标函数**：为进一步提升视觉真实感，可以将 VAE 目标与对抗损失结合（VAE-GAN 思路），在解码后的样本上使用判别器。该方法通常能让输出图像更锐利，但会引入额外的优化不稳定性与更多超参数。



讲义幻灯片：隐扩散模型（LDM）的配方。五步：取数据 → 编码成隐变量（对 VAE 取预测的均值）→ 得到尺寸小得多的隐变量数据集 → 在隐变量数据集上建立扩散模型 → 采样后解码回数据空间。要点：与之前完全相同的配方，只是数据集被变换了。 · 来源：MIT 6.S184 讲义

3.7. 注记 32：在隐空间建模

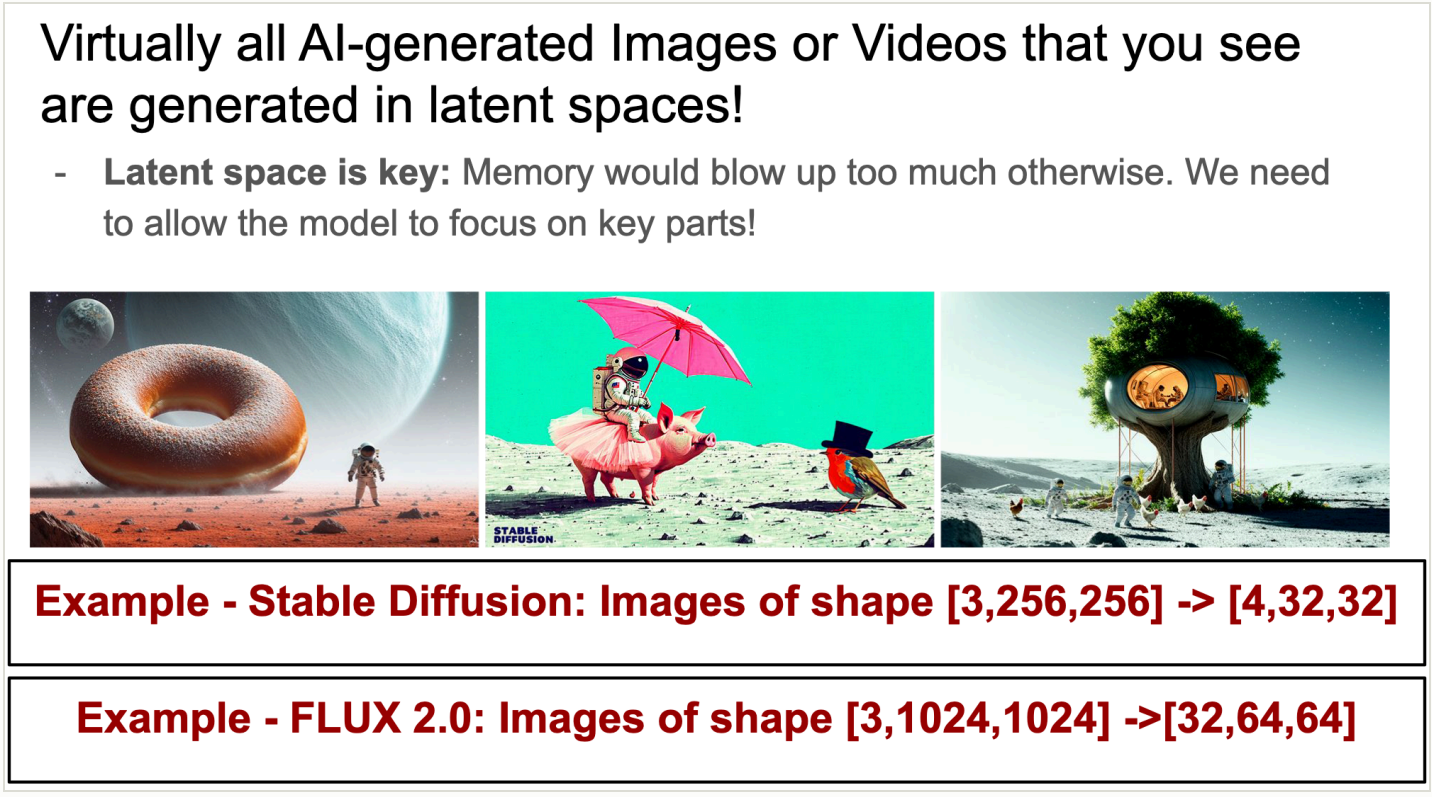
注记 32 · 在隐空间建模

训练隐空间生成模型时，我们沿用之前的训练流程，只是直接在隐空间进行计算。训练阶段：从 $\boldsymbol{x} \sim p_{\text{data}}$ 采样图像，再从 $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ 采样对应的隐变量 $\boldsymbol{z}$。推理阶段：从隐空间扩散模型或流模型中采样隐向量 $\boldsymbol{z}$，再通过 $\boldsymbol{x} = \boldsymbol{\mu}_{\text{mean}}(\boldsymbol{z})$ 解码（注意这里取均值而非随机采样，避免噪声带来的伪影）。校注：原文这里写作 $\boldsymbol{\mu}_{\text{mean}}$，按上下文应指前文的解码器均值 $\boldsymbol{\mu}_\theta$。

直观理解：训练良好的自编码器相当于过滤掉高频、无语义意义的细节，让生成模型可以“专注”学习对感知更重要的特征 [36]。在本文撰写时，几乎所有图像、视频生成的 SOTA 方法都采用**隐扩散范式**：在自编码器的隐空间内训练流模型或扩散模型 [36, 48]。

但有一点至关重要：**必须先训练好自编码器，再训练扩散 / 流模型**。模型最终性能同时取决于自编码器把图像压缩到隐空间、以及还原出高质量图像的能力。

更多关于 VAE 的讨论放在原讲义的附录 D（本文未收录）。



讲义幻灯片：几乎所有你见到的 AI 生成图像和视频都是在隐空间里生成的。隐空间是关键：否则内存会爆，需要让模型专注于关键部分。例：Stable Diffusion，[3,256,256] → [4,32,32]；FLUX 2.0，[3,1024,1024] → [32,64,64]。 · 来源：MIT 6.S184 讲义

04. 案例研究：Stable Diffusion 3 与 Meta Movie Gen

以下对应原讲义的 6.3 节。本节结尾我们简要分析两个大规模生成模型：面向图像生成的 Stable Diffusion 3，以及面向视频生成的 Meta Movie Gen Video [14, 33]。可以看到，这两个模型大量运用了本文介绍的各类技术，同时增加了架构上的增强，实现模型规模扩展，并且支持文本这类结构复杂的条件输入。

4.1. Stable Diffusion 3

Stable Diffusion 是一系列 SOTA 图像生成模型，也是最早一批将大规模隐式扩散模型用于图像生成的模型。强烈建议读者在线体验该模型（https://stability.ai/news/stable-diffusion-3）。

Stable Diffusion 3 使用本文所介绍的**条件流匹配损失目标**（见前几讲的算法 4）[4]。根据其技术报告，团队对比测试了多种流模型、扩散模型方案，发现流匹配效果最优。训练过程采用前几讲所述的无分类器引导训练（随机丢弃条件标签）。同时遵循原讲义 6.2 节（即本文第 03 节）的方案，在**预训练自编码器的隐空间**内完成训练；初代 Stable Diffusion 的一大贡献就是训练出效果优异的自编码器。

为增强文本条件控制能力，SD3 同时使用三类文本嵌入：CLIP 嵌入、谷歌 T5-XXL 预训练编码器输出的序列 token 嵌入等，实现思路参考文献 [3, 39]。

- CLIP 嵌入：提供对输入文本整体、粗粒度的语义表征；
- T5 序列嵌入：提供细粒度上下文信息，允许模型关注提示词内部各个具体元素。

为适配这种序列型上下文嵌入，研究者对扩散 Transformer 做扩展：注意力层不仅对图像 patch 做计算，同时接收文本嵌入做交互。把原本面向类别条件的 DiT 改造为支持序列上下文嵌入，这个改进后的架构称为**多模态 DiT（MM-DiT）**，结构见图 16。该系列最大版本模型参数量达到 80 亿。

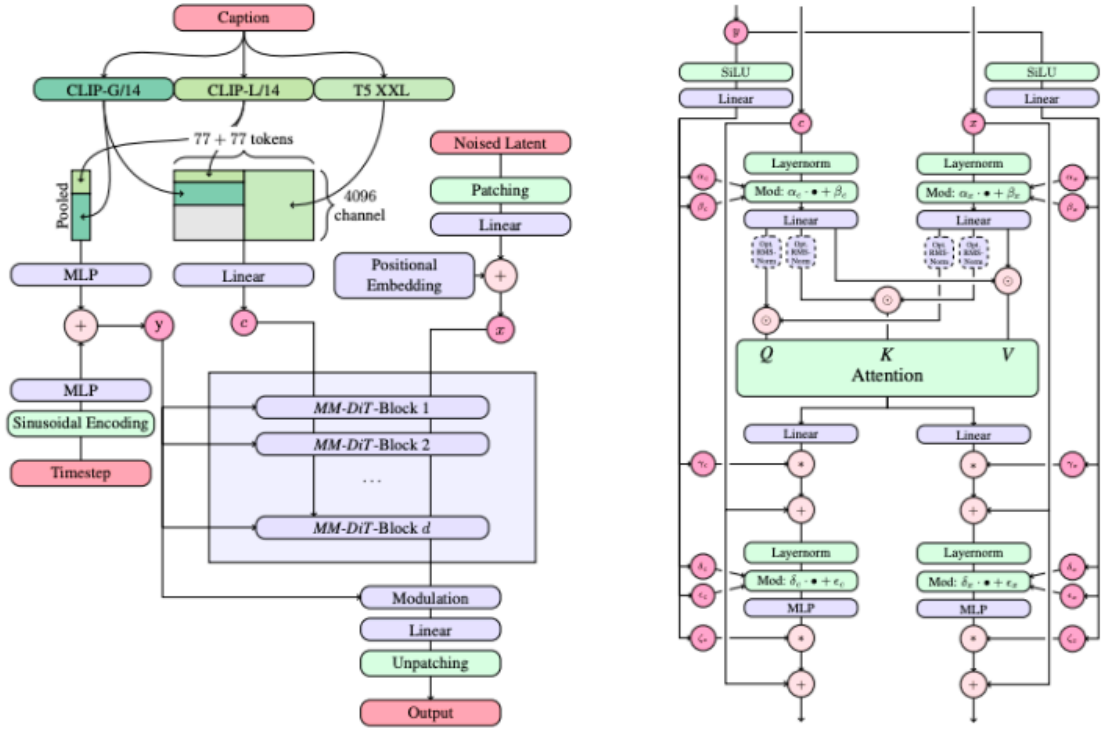


Figure 16: The architecture of the multi-modal diffusion transformer (MM-DiT) proposed in [14]. Figure also taken from [14].

图 16. [14] 提出的多模态扩散 Transformer（MM-DiT）架构，取自 [14]。 · 来源：文献 [14]，MIT 6.S184 讲义

采样阶段：采用欧拉数值求解方案，一共 50 步网络推理；无分类器引导权重设置在 2.0 ∼ 5.0。

4.2. Meta Movie Gen Video

接下来介绍 Meta 的视频生成模型 Movie Gen Video（https://ai.meta.com/research/movie-gen/）。视频数据 $\boldsymbol{x} \in \mathbb{R}^{T \times C \times H \times W}$，相比图像新增时间维度 T（即视频帧数）。视频模型的大量设计思路，本质就是把图像领域成熟技术（自编码器、扩散 Transformer 等）适配扩展，处理新增的时间维度。

Movie Gen Video 使用条件流匹配损失，采用直线调度方案：$\alpha_t = t$, $\sigma_t = 1 - t$。和 SD3 一致，模型运行在**冻结预训练自编码器的隐空间**。

补充

对于视频，自编码器降低显存开销的作用比图像场景更加关键；这也是目前绝大多数视频生成模型生成视频时长十分有限的重要原因。

作者提出**时间自编码器 TAE（Temporal Autoencoder）**处理时间维度：将原始视频 $\boldsymbol{x}_t' \in \mathbb{R}^{T' \times 3 \times H' \times W'}$ 映射为隐变量 $\boldsymbol{x}_t \in \mathbb{R}^{T \times C \times H \times W}$，时间、高、宽方向的压缩倍率均为 8 [33]，即

$$\frac{T'}{T} = \frac{H'}{H} = \frac{W'}{W} = 8$$

为支持长视频，提出时间分块策略：把完整视频切分为片段，每段分别编码，之后再将隐变量拼接融合 [33]。

主干网络 $\boldsymbol{u}_t^\theta(\boldsymbol{x}_t)$ 为类 DiT 架构，会同时在**时间维度与空间维度**对视频隐变量做分块 patchify。图像 patch 送入 Transformer，内部包含 patch 之间的自注意力，同时和语言模型嵌入做交叉注意力，设计思路类似 SD3 的 MM-DiT。

文本条件同样使用三类文本嵌入：

1. UL2 嵌入：实现细粒度文本语义推理 [47]；
2. ByT5 嵌入：捕捉字符级细节（例如提示词要求画面出现特定文字的场景）[50]；
3. MetaCLIP 嵌入：在图文共享嵌入空间做条件控制 [24, 33]。

该系列最大版本参数量达到 300 亿。想要更加详尽完整的介绍，可以阅读 Movie Gen 官方技术报告 [33]。

结语：**Stable Diffusion 3** 与 **Movie Gen** 对照

下表把第 04 节中散落的事实并排放在一起：

表 1. 两个案例所用零件的对照

项目	STABLE DIFFUSION 3	META MOVIE GEN VIDEO
数据	图像	视频 $\boldsymbol{x} \in \mathbb{R}^{T\times C\times H\times W}$，相比图像多了时间维度 T
训练目标	条件流匹配损失；技术报告对比多种流 / 扩散方案，流匹配最优；训练时随机丢弃条件标签（无分类器引导）	条件流匹配损失，直线调度 $\alpha_t = t$, $\sigma_t = 1 - t$
隐空间	预训练自编码器的隐空间	冻结的预训练自编码器的隐空间；TAE 把时间、高、宽都压缩 8 倍；长视频按时间分块编码后再拼接
主干网络	MM-DiT（图 16），最大版本 80 亿参数	类 DiT 架构，在时间与空间维度上 patchify；自注意力加与语言模型嵌入的交叉注意力，思路类似 MM-DiT；最大版本 300 亿参数
文本条件	CLIP 嵌入（整体、粗粒度）、T5 序列嵌入（细粒度）	UL2（细粒度语义推理）、ByT5（字符级细节）、MetaCLIP（图文共享嵌入空间）
采样	欧拉法，50 步网络推理；引导权重 2.0 ～ 5.0	正文未提

从表中可以看出：两个模型都是在冻结的预训练自编码器的隐空间里，用条件流匹配训练一个 DiT 类网络，并组合多种文本嵌入作为条件；视频主要是把图像上成熟的零件扩展到多出来的时间维度。